

Reto 4 – Comunicación Asincrónica y Eventos

Contexto

Este reto continúa el desarrollo del **sistema de onboarding y offboarding de empleados**. En retos anteriores se construyeron:

- Un servicio básico de empleados con Docker (Reto 1)
- Orquestación con Docker Compose, persistencia y comunicación REST entre servicios (Reto 2)

En este tercer reto se avanzará hacia una **arquitectura orientada a eventos**, incorporando:

- Un **message broker** como infraestructura de mensajería
- **Publicación de eventos** desde el servicio existente
- **Tres nuevos servicios** que reaccionan a eventos de forma autónoma
- El patrón **fan-out**: un solo evento dispara acciones en múltiples servicios

Objetivo

Incorporar comunicación asincrónica basada en eventos al sistema, de forma que los microservicios se comuniquen de manera desacoplada a través de un message broker. Al finalizar este reto, el sistema debe demostrar que un solo evento (como la creación de un empleado) desencadena reacciones automáticas e independientes en diferentes microservicios.

Requisitos generales

La solución desarrollada debe cumplir con los siguientes requisitos:

- Incorporar un **message broker** al `docker-compose.yml`
- El servicio de Empleados debe **publicar eventos** cuando se crea, actualiza o retira un empleado

- Implementar un **Servicio de Notificaciones** que consuma eventos y registre las notificaciones enviadas
- Implementar un **Servicio de Gestión de Perfiles** que consuma eventos y gestione el perfil del empleado
- Implementar un **Servicio de Gestión de Vacaciones** que gestione los períodos de vacaciones y publique sus propios eventos
- Todos los eventos deben respetar el **Catálogo de Eventos** del ecosistema: nombres, envelope y cargas útiles
- Los consumidores deben **tolerar eventos duplicados** (deduplicación por `id` del mensaje)
- Los estudiantes deben **investigar y seleccionar** el message broker a utilizar
- Documentar los endpoints de los nuevos servicios utilizando **OpenAPI (Swagger)**
- **Registrar los tres servicios nuevos en el API Gateway del Reto 3:** ninguno expone puertos al host
- Permitir el despliegue completo mediante un único comando

El punto de entrada único se mantiene. Los servicios que se crean en este reto entran al ecosistema **detrás del Gateway**, igual que los del Reto 3: usan `expose:` , nunca `ports:` , y se alcanzan por `http://localhost:8080/perfiles` , `/notificaciones` y `/vacaciones` . Agregue las rutas correspondientes al Gateway.

Diversidad tecnológica. El proyecto final exige **al menos 4 lenguajes de programación distintos**. Este reto es el momento natural para alcanzar ese número: implemente `perfiles-service` , `notificaciones-service` y `vacaciones-service` en lenguajes diferentes entre sí y distintos de los ya usados en `empleados-service` y `departamentos-service` . Documente en el README la tabla servicio ↔ lenguaje.

1. Message Broker

Investigación y selección

Cada equipo debe investigar al menos **tres** de las siguientes opciones de message broker y seleccionar **una** para su implementación:

Broker	Características clave
RabbitMQ	Protocolo AMQP, exchanges y colas, management UI, amplia documentación

Broker	Características clave
Apache Kafka	Alto throughput, persistencia de mensajes, ideal para streaming
Redis Streams	Ligero, bajo overhead, ideal si ya se usa Redis
NATS	Ultra-ligero, alta performance, cloud-native

Requisitos

- Incluir el broker como servicio en el `docker-compose.yml`
- Configurar la interfaz de administración del broker (si está disponible)
- Documentar en el README **por qué se eligió** ese broker sobre las alternativas

Ejemplo de configuración (RabbitMQ como referencia)

```
services:
  # ... servicios existentes del Reto 2 ...

  message-broker:
    image: rabbitmq:3-management
    ports:
      - "5672:5672"      # Puerto del broker
      - "15672:15672"    # Interfaz de administración
    environment:
      RABBITMQ_DEFAULT_USER: admin
      RABBITMQ_DEFAULT_PASS: admin
    networks:
      - microservices-network
```

Importante: Este es solo un ejemplo de referencia con RabbitMQ. Si elige otro broker, la configuración será diferente. El estudiante debe investigar y configurar el broker seleccionado.

2. Publicación de eventos desde el Servicio de Empleados

Evolución del servicio

El Servicio de Empleados (desarrollado en el Reto 2) debe evolucionar para **publicar eventos** cuando ocurran acciones relevantes:

Acción	Evento a publicar
Crear un empleado	<code>empleado.creado</code>
Actualizar un empleado	<code>empleado.actualizado</code>
Retirar un empleado	<code>empleado.retirado</code>

La carga útil de cada evento está definida en el [Catálogo de Eventos](#), secciones 3.1 a 3.3.

No invente nombres ni campos: el `auth-service` del Reto 5 y el resto del ecosistema dependen de que estos contratos se respeten al pie de la letra.

Comportamiento esperado

- Cuando se ejecuta `POST /empleados` exitosamente:
 - Se persiste el empleado en la base de datos (como en el Reto 2)
 - Se valida el departamento (como en el Reto 2)
 - Nuevo:** Se publica el evento `empleado.creado` en el broker
- Cuando se ejecuta `PUT /empleados/{id}` exitosamente:
 - Nuevo:** Se publica el evento `empleado.actualizado` en el broker
- Cuando se ejecuta `DELETE /empleados/{id}` (nuevo endpoint):
 - El empleado NO se borra físicamente.** Se cambia su `estado` a `RETIRADO` y se persiste la `fechaRetiro` (fecha y hora del retiro).
 - Nuevo:** Se publica el evento `empleado.retirado` en el broker

Baja lógica y auditoría

El retiro de un empleado es una **transición de estado**, no un borrado. Esto es una regla de dominio establecida en el Reto 00: *"Eliminar = marcar como RETIRADO, no se borra físicamente"*.

Debe implementarse además el endpoint de auditoría:

Método	Ruta	Descripción
GET	/empleados?estado=RETIRADO	Lista los empleados retirados
GET	/empleados?estado=RETIRADO&desde=2026-01-01&hasta=2026-06-30	Filtra los retiros por rango de fechas

La información retornada es de **solo lectura**: incluye la `fechaRetiro` de cada empleado.

Consideraciones

- La publicación del evento debe ocurrir **después** de que la operación en base de datos sea exitosa
- Si la publicación del evento falla, el servicio debe registrar el error pero **no revertir** la operación de base de datos
- El formato del evento (JSON) debe seguir el envelope del catálogo (`id` , `type` , `version` , `occurredAt` , `producer` , `data`)

3. Servicio de Notificaciones (nuevo servicio)

Un microservicio que **solo consume eventos**. Este servicio no es invocado por REST desde otros servicios, sino que reacciona autónomamente a los eventos del sistema.

Eventos que debe consumir

Evento	Acción
<code>empleado.creado</code>	Registrar y simular envío de email de bienvenida
<code>empleado.retirado</code>	Registrar y simular notificación de desvinculación
<code>vacaciones.programadas</code>	Registrar y simular confirmación del período de vacaciones

Simulación de notificaciones

Para simplificar, la "notificación" se simula mediante un **log estructurado** en la consola del servicio:

```
[NOTIFICACIÓN] Tipo: BIENVENIDA | Para: juan@empresa.com | Mensaje: "Bienvenido Juan Pé
[NOTIFICACIÓN] Tipo: DESVINCULACIÓN | Para: juan@empresa.com | Mensaje: "Su cuenta ha s
[NOTIFICACIÓN] Tipo: VACACIONES | Para: juan@empresa.com | Mensaje: "Sus vacaciones del
```

Registro de notificaciones

El servicio debe mantener un historial de notificaciones enviadas:

Método	Ruta	Descripción
GET	/notificaciones	Lista todas las notificaciones registradas
GET	/notificaciones/{empleadoId}	Lista notificaciones de un empleado específico

Estructura de una notificación

```
{
  "id": "string",
  "tipo": "BIENVENIDA | DESVINCULACION | VACACIONES",
  "destinatario": "string (email)",
  "mensaje": "string",
  "fechaEnvio": "datetime",
  "empleadoId": "string"
}
```

Deduplicación (obligatorio)

Un broker puede reentregar el mismo mensaje si el consumidor se cae antes de confirmarlo. Sin protección, el empleado recibiría dos correos de bienvenida.

Antes de procesar un evento, el servicio debe verificar si ya procesó ese `id` de mensaje; si es así, lo descarta y confirma sin repetir el efecto. Una tabla `eventos_procesados(id, procesadoEn)` es suficiente en esta etapa.

Verificación exigida: publique manualmente el mismo evento dos veces (desde la UI del broker) y demuestre que solo se registra **una** notificación.

El mecanismo formal (Inbox Pattern, DLQ, backoff exponencial) se estudia en el **Reto 29**.

Contenerización y base de datos

- Debe crear un Dockerfile para el servicio
- Debe tener su propia base de datos para el historial de notificaciones
- La configuración debe obtenerse de variables de entorno

Bonus (opcional): Implementar envío de email real utilizando un servicio SMTP (como Mailhog en Docker para pruebas locales).

4. Servicio de Gestión de Perfiles (nuevo servicio)

Un microservicio que **combina comunicación asincrónica y sincrónica**: consume eventos para crear datos automáticamente, y expone endpoints REST para que los datos puedan ser consultados y actualizados.

Eventos que debe consumir

Evento	Acción
<code>empleado.creado</code>	Crear un perfil por defecto para el nuevo empleado
<code>empleado.actualizado</code>	Sincronizar en el perfil los campos replicados del empleado
<code>empleado.retirado</code>	Archivar el perfil (marcarlo como archivado, no borrarlo)

Perfil por defecto

Cuando se recibe el evento `empleado.creado`, se crea automáticamente un perfil con los datos básicos:

```
{
  "id": "string",
  "empleadoId": "string",
  "nombre": "string (del evento)",
  "email": "string (del evento)",
  "telefono": "",
  "direccion": "",
  "ciudad": "",
  "biografia": "",
  "fechaCreacion": "datetime"
}
```

Endpoints requeridos

Método	Ruta	Descripción
GET	/perfiles/{empleadoId}	Consulta el perfil de un empleado
PUT	/perfiles/{empleadoId}	Actualiza el perfil (teléfono, dirección, biografía, etc.)
GET	/perfiles	Lista todos los perfiles

Respuestas esperadas

- **Consulta exitosa:** 200 OK con el perfil del empleado
- **Perfil no existe:** 404 Not Found con mensaje descriptivo
- **Actualización exitosa:** 200 OK con el perfil actualizado

Contenerización y base de datos

- Debe crear un Dockerfile para el servicio
- Debe tener su propia base de datos
- La configuración debe obtenerse de variables de entorno

5. Servicio de Gestión de Vacaciones (nuevo servicio)

Un microservicio que **produce eventos propios**. A diferencia de Notificaciones (solo consume) y Perfiles (consume + expone REST), este servicio es el primero del ecosistema que **origina una**

cadena de eventos que afecta a otro dominio: programar vacaciones terminará desactivando la cuenta del empleado, sin que Vacaciones sepa nada sobre cuentas ni autenticación.

Esa es exactamente la propiedad que hace valioso el desacoplamiento por eventos: el `vacaciones-service` publica "este período comenzó" y no le importa quién reacciona.

Endpoints requeridos

Método	Ruta	Descripción
POST	/vacaciones	Programa un período de vacaciones para un empleado
GET	/vacaciones/{id}	Consulta un período por su identificador
GET	/vacaciones?empleadoId={id}	Lista los períodos de un empleado
GET	/vacaciones	Lista todos los períodos registrados
DELETE	/vacaciones/{id}	Cancela un período que aún no ha iniciado

Estructura de un período de vacaciones

```
{
  "id": "V-2026-0042",
  "empleadoId": "E001",
  "fechaInicio": "2026-03-15",
  "fechaFin": "2026-03-30",
  "estado": "PROGRAMADA",
  "fechaCreacion": "2026-03-01T10:00:00Z"
}
```

Estados del período: `PROGRAMADA` → `EN_CURSO` → `FINALIZADA` , o `CANCELADA` .
En este reto el período solo alcanza el estado `PROGRAMADA` ; las transiciones automáticas por fecha se implementan en el **Reto 5** con el scheduler.

Validaciones requeridas

Al programar un período, el servicio debe rechazar la petición con `400 Bad Request` y un mensaje descriptivo si:

- 1. **Fechas incoherentes:** `fechaFin` no es posterior a `fechaInicio` .

2. **Fechas en el pasado:** `fechaInicio` es anterior a la fecha actual.
3. **Solapamiento:** el empleado ya tiene un período `PROGRAMADA` o `EN_CURSO` que se cruza con el rango solicitado. La respuesta debe incluir el período en conflicto.
4. **Empleado inexistente:** el `empleadoId` no corresponde a un empleado registrado.

Decisión técnica requerida. La validación 4 exige saber si el empleado existe. Tiene dos opciones y debe justificar la elegida en el README:

- **(a) Consulta síncrona REST** al `empleados-service`, como hizo el Reto 2 con departamentos. Simple, pero acopla los dos servicios y falla si Empleados está caído.
- **(b) Réplica local por eventos:** el `vacaciones-service` consume `empleado.creado` y `empleado.retirado` y mantiene su propia tabla mínima de empleados válidos. Más trabajo, pero el servicio queda autónomo. Ambas son defendibles. Lo que se evalúa es que entienda el intercambio: **¿prefiere disponibilidad o consistencia inmediata?**

Evento que debe publicar

Acción	Evento
Programar un período exitosamente	<code>vacaciones.programadas</code>

Carga útil definida en el [Catálogo de Eventos](#), sección 3.8.

Importante: este evento **no desactiva la cuenta**. Solo anuncia que el período quedó registrado, y el `notificaciones-service` lo usa para confirmarle las fechas al empleado. La desactivación ocurre el día que arrancan las vacaciones, con el evento `vacaciones.iniciadas` que se implementará en el **Reto 5**.

Contenerización y base de datos

- Debe crear un Dockerfile para el servicio
- Debe tener su propia base de datos
- La configuración debe obtenerse de variables de entorno

6. Pruebas del Sistema

Verificar el funcionamiento completo del flujo asincrónico:

Flujo de prueba sugerido

1. Iniciar todos los servicios: `docker-compose up --build`
2. Verificar que el broker está activo y accesible (management UI)
3. Crear un departamento (como en el Reto 2):

```
curl -X POST http://localhost:8080/departamentos \
  -H "Content-Type: application/json" \
  -d '{"id": "IT", "nombre": "Tecnología", "descripcion": "Departamento de TI"}'
```

4. Crear un empleado:

```
curl -X POST http://localhost:8080/empleados \
  -H "Content-Type: application/json" \
  -d '{"id": "E001", "nombre": "Juan", "apellido": "Pérez",
    "email": "juan.perez@empresa.com", "numeroEmpleado": "EMP-2026-001",
    "cargo": "Desarrollador Senior", "area": "Tecnología",
    "departamentoId": "IT", "fechaIngreso": "2026-03-01"}'
```

5. **Verificar que el evento fue procesado:**

- Consultar el perfil creado automáticamente: `GET http://localhost:8080/perfiles/E001`
- Consultar la notificación registrada: `GET http://localhost:8080/notificaciones/E001`
- Revisar los logs del servicio de notificaciones para ver la notificación simulada

6. Actualizar el perfil del empleado:

```
curl -X PUT http://localhost:8080/perfiles/E001 \
  -H "Content-Type: application/json" \
  -d '{"telefono": "3001234567", "ciudad": "Armenia", "biografia": "Ingeniero de si
```

7. **Programar vacaciones y verificar la confirmación asincrónica:**

```
curl -X POST http://localhost:8080/vacaciones \
  -H "Content-Type: application/json" \
  -d '{"empleadoId": "E001", "fechaInicio": "2026-06-15", "fechaFin": "2026-06-30"}'
```

```
# Verificar que llegó la notificación de tipo VACACIONES
curl http://localhost:8080/notificaciones/E001
```

8. **Verificar las validaciones de vacaciones** (todas deben responder `400`):

```
# Fechas incoherentes
curl -X POST http://localhost:8080/vacaciones -H "Content-Type: application/json" \
  -d '{"empleadoId": "E001", "fechaInicio": "2026-06-30", "fechaFin": "2026-06-15"}'

# Solapamiento con el período ya creado
curl -X POST http://localhost:8080/vacaciones -H "Content-Type: application/json" \
  -d '{"empleadoId": "E001", "fechaInicio": "2026-06-20", "fechaFin": "2026-07-05"}'

# Empleado inexistente
curl -X POST http://localhost:8080/vacaciones -H "Content-Type: application/json" \
  -d '{"empleadoId": "NO-EXISTE", "fechaInicio": "2026-08-01", "fechaFin": "2026-08-01"}'
```

9. **Verificar la deduplicación:** desde la UI de administración del broker, publique manualmente dos veces el mismo mensaje `empleado.creado` (mismo `id` de envelope) y confirme que solo se registra **una** notificación y **un** perfil.

10. Retirar un empleado y verificar la baja lógica:

```
curl -X DELETE http://localhost:8080/empleados/E001

# El empleado NO desaparece: cambia de estado
curl http://localhost:8080/empleados/E001 # estado: RETIRADO, con fechaRet
curl "http://localhost:8080/empleados?estado=RETIRADO"

# Verificar notificación de desvinculación y perfil archivado
curl http://localhost:8080/notificaciones/E001
curl http://localhost:8080/perfiles/E001 # archivado
```

11. Reiniciar los contenedores y verificar que los datos persisten

Entregables

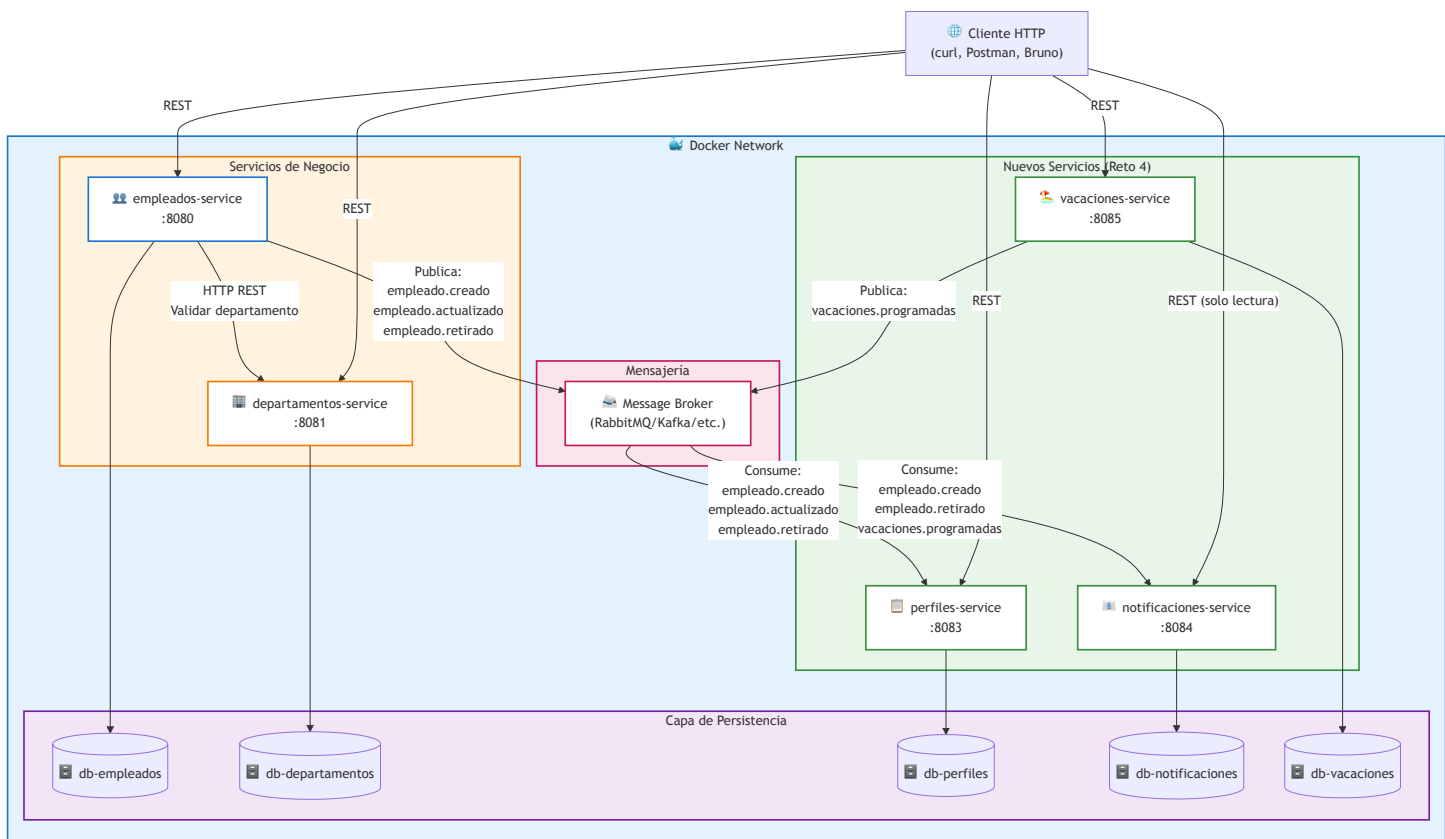
- Versione adecuadamente su código en repositorios de GitHub
- Cree un [README.md](#) actualizado que incluya:
 - Justificación de la elección del message broker
 - Tabla **servicio** ↔ **lenguaje de programación** (mínimo 4 lenguajes distintos en el ecosistema)
 - Instrucciones de despliegue del sistema completo

- Documentación de los eventos implementados, referenciando el [Catálogo de Eventos](#)
- Justificación de la estrategia elegida para validar la existencia del empleado en vacaciones-service (consulta síncrona vs. réplica por eventos)
- Evidencia de la prueba de deduplicación (evento publicado dos veces → un solo efecto)
- Instrucciones de prueba para verificar el flujo asíncronico

Consideraciones

- Cada servicio debe tener su propio Dockerfile
- Cada servicio debe tener su propia base de datos
- El servicio de Notificaciones debe ser **puramente reactivo** (no es llamado por otros servicios via REST)
- El servicio de Perfiles combina ambos estilos: reactivo (consume eventos) + REST (expone endpoints)
- El servicio de Vacaciones es **productor**: expone REST hacia RRHH y origina eventos que otros dominios consumirán
- Los nombres y cargas útiles de los eventos **no son negociables**: el Reto 5 construye sobre ellos. Respete el catálogo

Diagrama de Arquitectura Esperada



Criterios de Evaluación

El reto se evalúa sobre **5 puntos**, distribuidos de la siguiente forma:

#	Elemento	Valor	Aspectos a evaluar
1	Message Broker	0.5	Configuración correcta en compose, investigación documentada, justificación de la elección
2	Publicación de eventos y baja lógica	1.0	El servicio de Empleados publica empleado.creado , empleado.actualizado y empleado.retirado respetando el envelope del catálogo. El retiro es una transición a RETIRADO con fechaRetiro , no un borrado. Endpoint de auditoría con filtro por rango de fechas.
3	Servicio de Notificaciones	1.0	Consumo de los tres eventos, registro de notificaciones, log de simulación, endpoints de consulta y documentación

#	Elemento	Valor	Aspectos a evaluar
			OpenAPI. Deduplicación por id de mensaje demostrada.
4	Servicio de Perfiles	1.0	Consumo de empleado.creado , empleado.actualizado y empleado.retirado . Creación de perfil por defecto, sincronización, archivado. Endpoints REST funcionales y documentación OpenAPI.
5	Servicio de Vacaciones	1.0	CRUD funcional. Las cuatro validaciones implementadas (fechas incoherentes, fechas pasadas, solapamiento, empleado inexistente). Publica vacaciones.programadas según el catálogo. Estrategia de validación del empleado justificada. Documentación OpenAPI.
6	Pruebas y documentación	0.5	Flujo completo demostrable, tabla servicio ↔ lenguaje (mínimo 4 lenguajes), README actualizado, correcto funcionamiento de Swagger UI

Nota: La documentación ([README.md](#)) y el versionamiento adecuado en GitHub son requisitos transversales que afectan la evaluación de cada elemento.